

Vulcan Activity Tracker

The Athletic Activity Tracker for PennWest California Students.



Design Document

Team Members:

Margo Bonal

John Gerega

Luke Ruffing

Professor:

Dr. Weifeng Chen

Course: Software Engineering Project I

School: Pennsylvania Western University

Instructor Comments/Evaluation

Table of Contents

Abstract.....	5
Description of the Document.....	6
Purpose and Use:.....	6
Ties to the specification document:	6
Intended Audience:	6
Project Block System Diagram	7
Design Details	8
System Modules and Responsibilities	8
System Overview	8
Overview	8
Architectural Diagram Description.....	9
Architectural Diagram	10
Module Cohesion.....	11
Module Coupling.....	11
Design Analysis.....	12
Data Flow Diagram.....	13
Design Organization	14
Detailed Tabular Description of Classes and Object	14

Functional Descriptions: 18

Input / Output / Return Parameters & Types..... 18

Modules Used 23

Files Accessed 24

Real-time Requirements 24

Messages 25

Narrative/PDL..... 25

PDL Outline 26

Decision: Programming language / Reuse / Portability..... 27

Implementation Timeline..... 27

Design Testing 28

List of Resources..... 29

Appendix I: Schematic and Bill of Materials 30

Possible Hosting Service Options: 30

Schematic of Toolset:..... 30

Appendix II: Team Details 31

Appendix III: Workflow Authentication 32

Appendix IV: Report from Writing Center 33

Abstract

This paper is designed to be read by the software developers of the Vulcan Activity Tracker. It will lay out the development side designs and requirements of this application. This application will allow users to log in using a PennWest email address and a unique password, manually track an exercise they have done, and then store their exercise in a backend database. The developers will use the information in this document to gain insight on their best process to handle the project. The details list the interaction between the frontend and backend elements of the application, as well as the programming languages that will be used to achieve this.

Description of the Document

Purpose and Use:

The purpose of this document is to give software development guidance to the architecture of the software. This document will also show in detail the programming language and functions needed to make sure the application runs smoothly. It will be used by the software developers as a way to ensure the project stays on track without too many disruptions or delays.

Ties to the specification document:

This document extends what the specification document shows and adds components relevant to the software development side. This document will show what programming language will be used and what style of programming architecture will be implemented into the project.

Intended Audience:

This document's target audience are both the software developers and designers as there are many technical terms and phrases within. This document is not meant to be read by the client, but it may be in the best interest of them to have a technically capable third-party member to review the document and ensure that it aligns with the client's requests. Software developers and designers should use this document to better understand the scope of the Vulcan Activity Tracker, including the application's modules, data flow, variables, functionality, architecture and system requirements.

Project Block System Diagram

The Vulcan Activity Tracker will consist of 2 main software components, Client (Frontend) and Server (Backend) Sides. Client Side will be user interface that is accessible by internet hosting. Server Side will be the driving functionality of the application that will be designed to handle data propagation and database storage. The UI will prepare and send fitness activity data to the Backend Server. The Server will then clean the data and prepare it for database entry. When received by database handling logic, the fitness activity data is then sent into and stored by the MSSQL database.

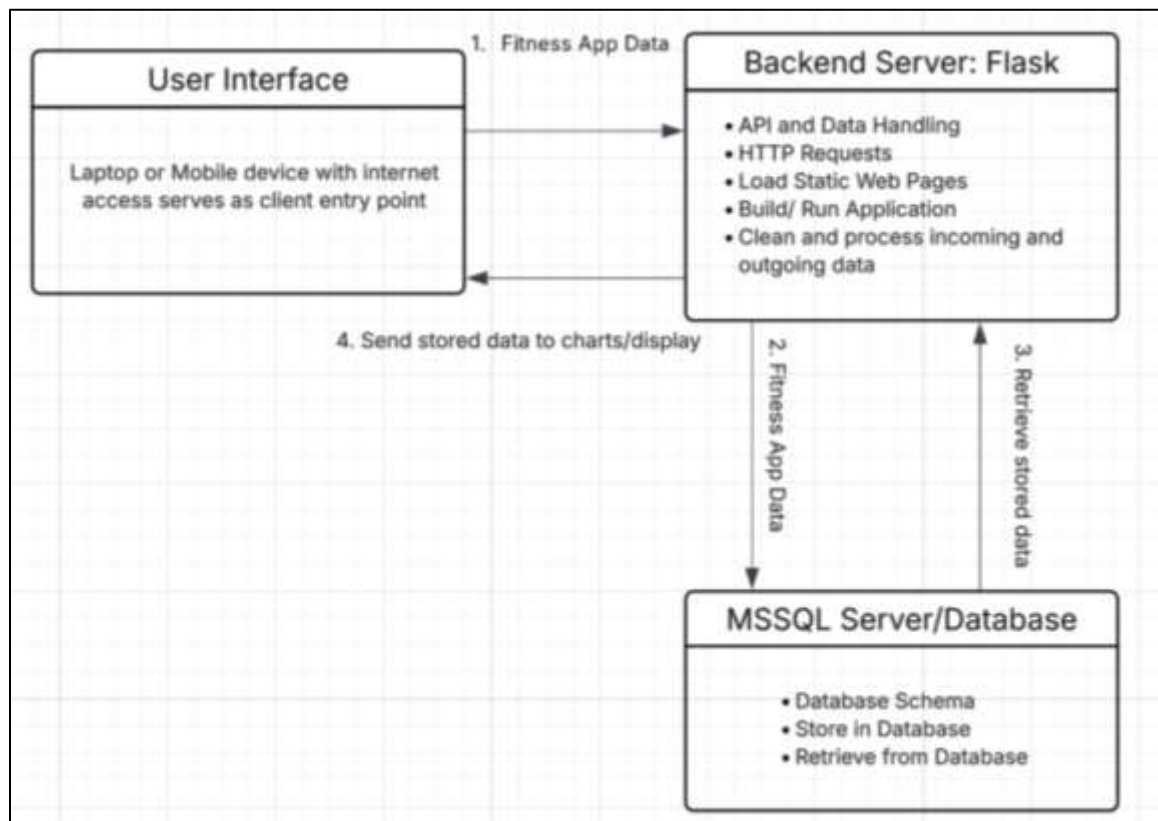


Figure 1: Block Diagram

Design Details

System Modules and Responsibilities

The Vulcan Activity Tracker will be composed of many system modules that hold the responsibility of data transfer, propagation, display, and other driving factors that will bring the project from concept to actuality. Modules will be designed with an object-oriented approach, aiding reusability and time efficiency. All team members are trained on the object-oriented method; however, team meetings will be held to agree on cohesive structure design.

System Overview

The following system overview will represent the design strategy needed to incorporate all fitness and user data for the Vulcan Activity Tracker. The Vulcan Activity Tracker has many pieces of information that it will handle. This includes user demographics for profile registration/passwords and fitness activity information. The application also will store activity environment information of running paths and gym amenities. Data storage will be the most complex but key feature of the Vulcan Activity Tracker. Cleaning and preparing the data for database entry must happen initially. Applying validation checks of data entry fields will aid in eliminating cyberattacks. By implementing a private database designed only for application use, all data can be stored and used without pollution of other external sources.

Overview

The Vulcan Activity Tracker, once completed, will be available for California campus students to access through URL link. This link can be accessed through either laptop, PC, or mobile devices with updated web-browser. Upon entry to the web-based application, a student

will be prompted to login with campus email address. This email will be validated with proper pennwest.edu suffix and the student will be asked to make a custom password for entry.

Once verified, a campus student can then access the Vulcan Activity Tracker's amenities. Users will be able to view their profile, add a new workout, look at past fitness activities, and participate in group activities. Users will additionally be able to view the platform's leaderboard. All fitness and user data entered will be stored in the database and any calculations performed. Upon session, stored data will also be pulled from storage to fill all graphical features such as tables and graphs. Overall design strategy is proposed for usability, cohesive design, and functional features with the student user in mind.

Architectural Diagram Description

The following diagram, Figure 2, represents the Architectural Design of the Vulcan Activity Tracker. The diagram follows the users' access to the application and the appropriate actions that are triggered. The Web Application is the primary access entry point for both developer testing and end goal of user participation. The web application itself will support both data transmission and entry from the system to the server. It will be designed with receiving API connections in the form of HTTP request methods, GET and POST. The next state in the architectural diagram is the internet. This is a crucial layer represented below, since internet access both hosts the application and provides connection between multiple users. The final structure represented below is the Backend/Server. This component will be the most complex factor of development since it handles HTTP requests, port specification, data handling, and database schema configurations.

Architectural Diagram

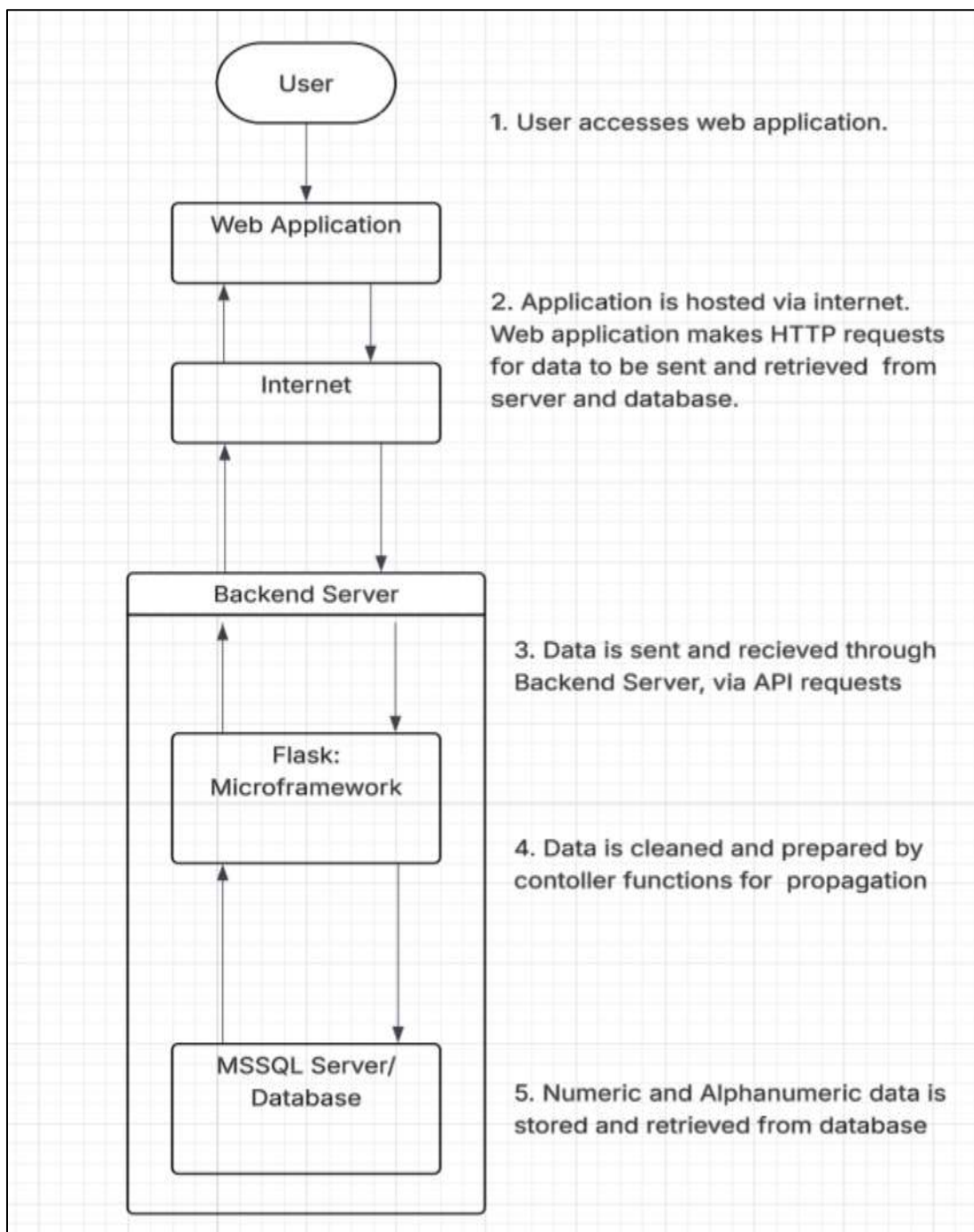


Figure 2: Architectural Diagram

Module Cohesion

Module cohesion refers to the degree to which elements inside a module belong together. Each of the modules in the Vulcan Activity Tracker have a well-defined responsibility resulting in high cohesion throughout the system. The following points represent the systematic approach designed to meet cohesion needs.

- The Login & Authentication Module: This module focuses solely on validating the login credentials of users.
- The Activity Management Module: This module handles only the creation, validation, removal, and retrieval of activities.
- The Group Management Module: This module handles only the management of club/group memberships and roles.
- The Leaderboard Module: This module focuses only on processing and displaying the rankings within different parts of the Vulcan Activity Tracker.
- The Database Module: This module solely manages the storage and retrieval of data items into and out of the database.

Module Coupling

Module Coupling refers to the interdependence between system modules and components. The modules within the Vulcan Activity Tracker have a low coupling to reduce the dependencies modules have on each other. The following points represent the systematic approach designed to meet coupling needs.

- The Login & Authentication Module: This module interacts with the Database Module to verify the user's login credentials.

- The Activity Management Module: This module interacts with the Database Module to store and retrieve activity data.
- The Group Management Module: This module interacts with the Database Module to store and retrieve group data.
- The Leaderboard Module: This module interacts with the Database Module to store and retrieve leaderboard data.
- The Database Module: This module interacts with all the previous modules to send data back and forth to the database server.

Design Analysis

The diagrams included in this Design document are to help both clients and developer roles to visualize the finished product's operation. Since the Vulcan Activity Tracker is designed for multiple users to operate and interact with, it is important that data flows properly from functions in a timely matter. With use case in mind, data throttling and database management time will need to be evaluated. The following Data Flow Diagram represents the various data components that will be migrating from user entry fields to database entry. Likewise, the data must be able to transport or flow the other direction from database to user interfacing as well. Careful architecture and design will be implemented to avoid data processing deadlocks or critical regions. During each process and checkpoint, testing will be performed on each module individually and as a system overall to ensure that development design is accurately depicting the provided documentation.

Data Flow Diagram

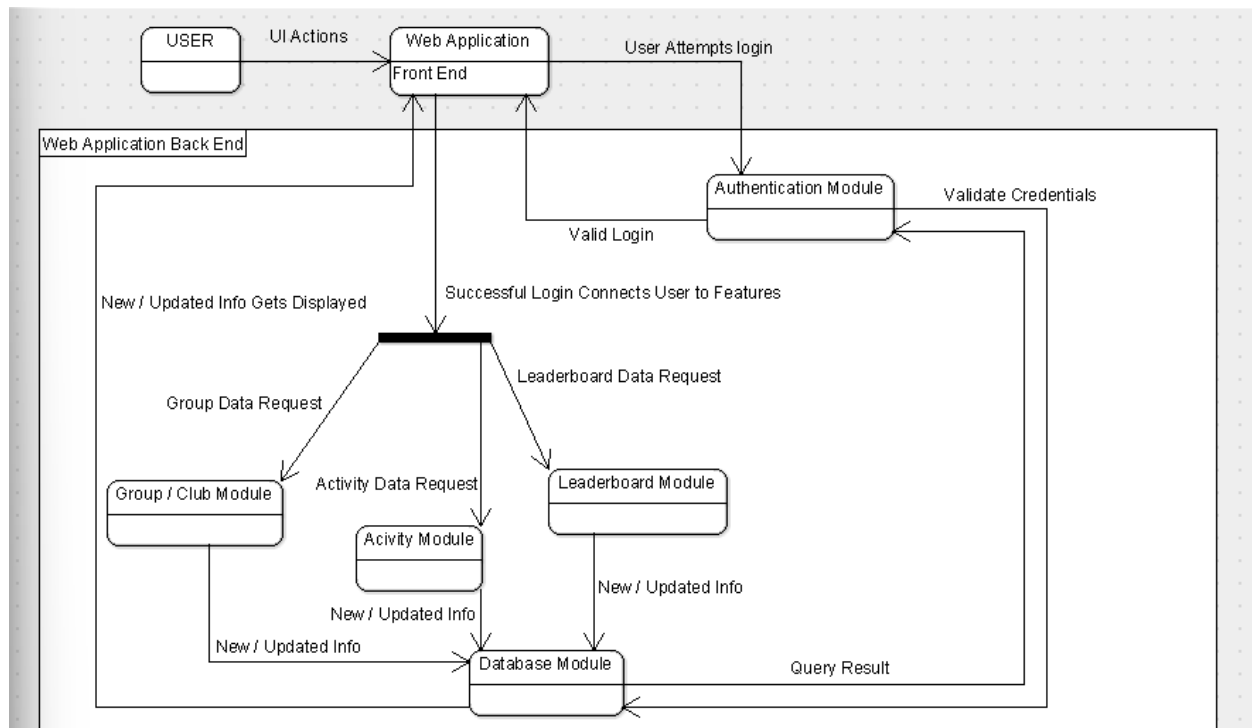


Figure 3: Data Flow Diagram

Figure 3 models the primary data flow of the Vulcan Activity Tracker web application. The process begins with the user attempting to login on the web applications front end UI, the login credentials entered by the user are passed to the authentication module for validation comparing the credentials to valid credentials in the database. Upon a successful login the home screen of the activity tracker is displayed on the front end to the user, and they can access all the functional modules, including Activity Management, Group/Club Management, and Leaderboards features. Each module individually communicates with the database module to store, retrieve or update information. Responses from the database module are then updated and presented to the user via the front-end web application.

Design Organization

Detailed Tabular Description of Classes and Object

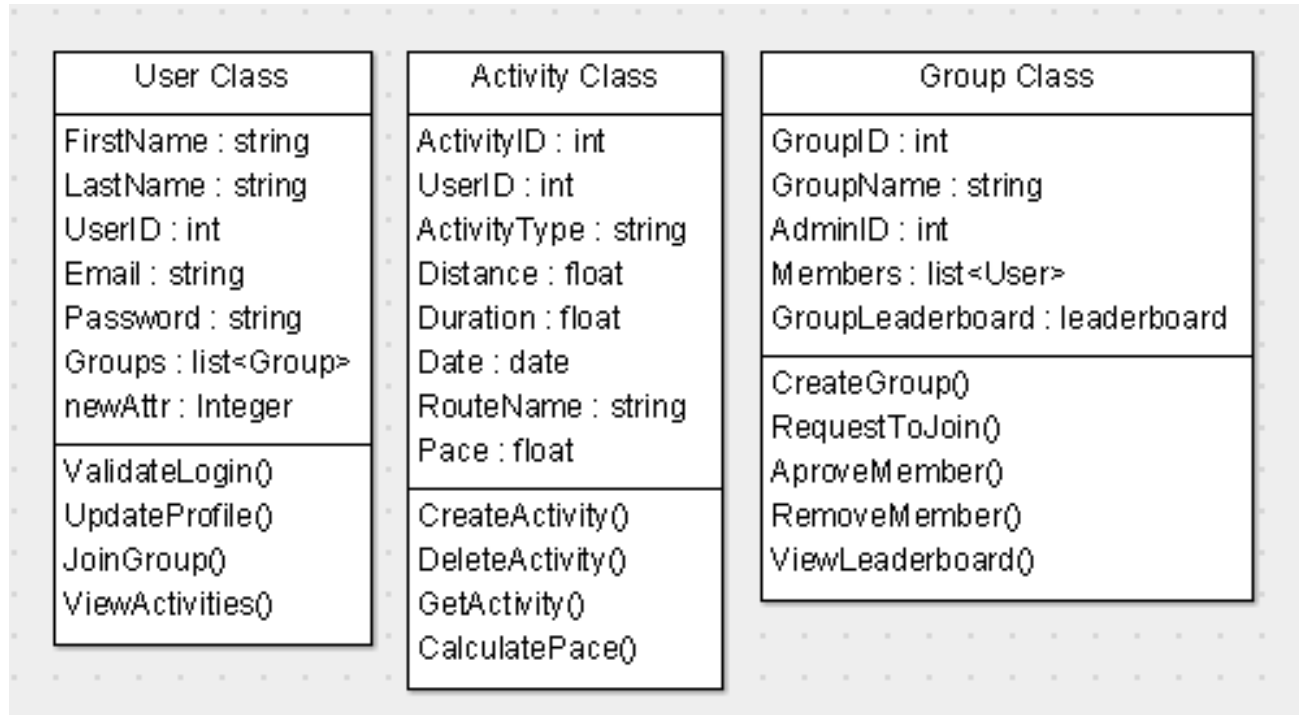


Figure 4: Overview of Classes

The above figure shows an overview of the 3 main classes that the Vulcan Activity Tracker makes use of. Every user with access to the web application will be assigned an instance of the user class. User demographics and login information is processed and structured by this class. The user can additionally create many instances of both the activity and group classes through the graphical user interfacing that allows activity and group assignment. This modular design aids in reuse during development and clarity during post-delivery maintenance. By implementing an object-oriented approach, time efficiency can be reached through session time compilation.

Class name:	User Class
Data members:	FirstName : string LastName : string UserID : int Email : string Password : string Groups : list<Groups>
Class member functions:	ValidateLogin() UpdateProfile() JoinGroup() ViewActivities()
Constraints:	Email data member must contain a “@pennwest.edu” email FirstName and LastName data members cannot be empty. The Password data member must hold a password that meets minimum security requirements. The UserID data member must hold a unique ID
Description:	The User Class holds the data needed for each user. It will hold the first name, last name, email and password that the user used to sign up, after sign up each user will be given a User ID which is an integer used to link the user to their activities and groups.

User class member function descriptions:

- ValidateLogin() – Verifies user credentials
- UpdateProfile() – Lets users modify their profile fields
- JoinGroup() – Sends request to join a certain group
- ViewActivities() – Retrieves the users added activities

Class name:	Activity Class
Data members:	ActivityID : int UserID : int ActivityType : string Distance : float Duration : float Date : date RouteName : string Pace : float
Class member functions:	CreateActivity() DeleteActivity() CalculatePace() GetActivity()
Constraints:	ActivityID data member must hold a unique ID UserID data member must hold a valid UserID ActivityType, Distance, Duration, and Date data members must be filled in. Pace and RouteName data members are optional
Description:	The Activity Class holds the data needed for each activity. As soon as the create activity button is pushed an activity object will be created giving the object a unique activity ID and storing the ID of the user creating the activity. Once created, the activity will ask for the input of activity type, distance, duration, date, route name (Optional), and Pace (Optional). Once all the needed data is added to the activity it will be stored in the database.

Activity class member function descriptions:

- CreateActivity() – Adds a new activity to the database
- DeleteActivity() – Removes an activity from the database
- CalculatePace() – Calculates the pace if distance and duration of an activity exist
- GetActivity() – Retrieves Activities

Class name:	Group Class
Data members:	GroupID : int GroupName : string AdminID : int Members : list<User> GroupLeaderboard : leaderboard
Class member functions:	CreateGroup() RequestToJoin() ApproveMember() RemoveMember() ViewLeaderboard()
Constraints:	GroupID data member must hold a unique ID AdminID data member must hold a valid UserID GroupName data member must be filled in
Description:	The Group Class holds the data needed to create and manage each group. Once a group is created by a user the group object will receive a unique group ID. The user will then input an appropriate group name and become the group's administrator. Each group object will also hold the Admin ID (ID of user that created the group), a list of the members, and a group leaderboard that tracks group performance.

Group class member function descriptions:

- CreateGroup() – Creates new group and makes creator the administrator of group
- RequestToJoin() – Sends a request to join to an admin
- ApproveMember() – Admin approved member request
- RemoveMember() – Admin removes user from group
- ViewLeaderboard() – Displays the group leaderboard stats

Functional Descriptions:

Input / Output / Return Parameters & Types

Activity Class

CreateActivity():

Input: CreateActivity will take in user data that is placed into a list

Output: The function will output a success or error message depending on whether or not the user enters all required data

Return Parameters: The only values that will be returned will be exceptions thrown during error checking, i.e. missing required data

Types: The user list will contain strings, integers, and Python's datetime type imported from the datetime library

CalculatePace():

Input: This function will take the pace input and duration input if the activity they are entering requires those fields

Output: The function will calculate the pace of the activity and upload it into the database

Return Parameters: The only values that will be returned will be exceptions thrown during error checking, i.e. missing data

Types: The user input will be a Python time, imported from the datetime library

GetActivity():

Input: User will enter the activity name they are looking for, which will be a string

Output: The function will output the activity the user has searched for

Return Parameters: The only values that will be returned will be exceptions thrown during error checking, i.e. missing data or activity not found in database

Types: The user input will be a string

DeleteActivity():

Input: The function will receive the name of the activity that the user wishes to delete

Output: The function will output a success or error message depending on whether or not the user enters all required data

Return Parameters: The only values that will be returned will be exceptions thrown during error checking, i.e. missing required data

Types: The input will be a string of the activity name they wish to delete

Group Class

CreateGroup(): Creates new group and makes creator the administrator of group

Input: Input will consist of group information, such as name, type of group, whether the group is private, and a designated administrator

Output: The function will output a success or error message depending on whether or not the user enters all required data

Return Parameters: The only values that will be returned will be exceptions thrown during error checking, i.e. missing required data

Type: The input will consist of strings for each field of information required to create a group

RequestToJoin() – Sends a request to join to an admin

Input: There will be a optional input, which will be the user's reason as to why they would like to join the group. The user will have to enter their username that goes along with the request

Output: A message will be displayed saying the request has been sent for review by an administrator of the group

Return Parameters: The only values that will be returned will be exceptions thrown during sending of the request, which in this case could be a database error or an error checking exception of the user not entering their username.

Type: Optional input will be a string, user's username will be extracted as a string

ApproveMember() – Admin approved member request

Input: The function will take in the admin's answer to the request, which will be to approve or to deny access

Output: The function will output a message depending on admin's answer to the request, if the admin approves the request, the user will be added to the group. If the admin denies the request, a message will be sent to the user denied.

Return Parameters: The values returned will be the messages returned during error exceptions, which could be an error in adding the user to the group

Type: The input will be a flag sent to the program based on the admin's response to the request

RemoveMember() – Admin removes user from group

Input: Function will receive the username or name of the member to be removed from the group

Output: Message of success will be displayed when the desired member is successfully removed from the group

Return Parameters: Values returned will be messages during error checking, which in this case could be an error in finding the member or removing the member

Type: Desired member's username will be passed in as a string

ViewLeaderboard() – Displays the group leaderboard stats

Input: User will press a button that will call this function

Output: Leaderboard for only the group will be displayed, with the user being able to filter other aspects of the leaderboard

Return Parameters: The only value that will be returned is messages returned from error checking

Type: Function is triggered by a button push

User Class

ValidateLogin() – Verifies user credentials

Input: User will enter an email and password. This email will be checked to see if it contains @pennwest.edu at the end. The program will then run a security check on the credentials

Output: User will either be redirected to the application or will have a message displayed that their credentials are wrong

Return Parameters: Values returned by the function will return a message that user cannot be logged in or that there is a missing field that was detected in error checking

Type: Username and password will be strings

UpdateProfile() – Lets users modify their profile fields

Input: User will go into their profile section and make their changes that they desire

Output: Message will be displayed that the changes were successful

Return Parameters: Values returned by this function include a success message or a message dealing with error checking, which could include missing information in a field

Type: User will be modifying strings in their profile

JoinGroup() – Sends request to join a certain group

Input: User presses button to request to join a group, along with optional reason why they would like to join

Output: A message saying the request was successfully sent will be displayed to the user

Return Parameters: Values returned by this function will be messages due to error checking, which in this case could be an error in sending the request to the proper administrator of the group

Type: Optional reason to join the group is a string, request is a button

ViewActivities() – Retrieves the users added activities

Input: User will press the button to retrieve their activity history

Output: A list of the user's activities will be displayed

Return Parameters: Values returned by this function will be messages produced by error checking, which in this case could be with retrieving the activities from the database

Type: User will push a button to activate this function

Modules Used

The Vulcan Activity Tracker is organized into an architecture that is consistent with Figure 2. At the top is the web application which interacts with the backend server through the internet. Within the backend server the following modules are implemented.

- The Login & Authentication Module: This module validates a user's PennWest credentials.
- The Activity Management Module: This module handles gathering and storing activity information.
- The Group Management Module: This module handles the management of club/group participation.

- The Leaderboard Module: This module processes and displays rankings within different parts of the Vulcan Activity Tracker.
- The Database Module: This module handles communication between the Vulcan Activity Tracker and its MSSQL database server.

Files Accessed

Physical file storage has been identified as a system vulnerability and out of scope for the Vulcan Activity Tracker. All data will be stored in the SQL database. This is to avoid vulnerabilities of locally or session stored data. Additionally, this avoids complex logic of file validation of uploading or downloading files to or from the system. SQL Server access will provide timely data lookup and table propagation for both login and application features.

Real-time Requirements

The real-time requirements of the Vulcan Activity Tracker are primarily internet access and hosting. WIFI connection is needed for internet access without cellular data service provider or for device access without SIM connectivity. The Vulcan Activity Tracker is therefore reliant on internet access for operation. Measures will be taken to avoid deadlock situations where processes attempt to use the same key functions at once. Also, during development, careful consideration will be aimed towards overall time efficiency. This will be seen in efficiently loading static interface pages into the server, reducing rendering time. Additionally, server injections will be analyzed for efficiency to limit all actions that have potential of lag for users. A final real-time requirement note is that potential losses of internet connection could have negative effects on the completion of data storage.

Messages

The following tables represent the messages that are sent and received by the Vulcan Activity Tracker's client and server. Current up to date HTTPS and Internet protocols will be followed to achieve application layer integration on the network. The following codes will aid in both development processes and debugging search. The Vulcan Activity Tracker developers will be appropriately trained on server status code to streamline the process. Both HTTP and SQL type messages are included below.

HTTP Status	Code
Informational Response	100-199
Successful Response	200-299
Redirection Message	300-399
Client Error Response	400-499
Server Error Response	500-599

SQL Error	Code
Syntax Correctness	-104
DML Correctness	-117
Schema Awareness	-204
Data Integrity	-407
Security & Permissions	-551

Narrative/PDL

The following narrative/PDL simulates a functional compilation and process of the Vulcan Activity Tracker from login until logout. Modules are loaded and processed upon graphical interfacing requests. Data is appropriately handled upon action request. During each step, data is appropriately cleaned and normalized for database entry. As shown below, the Vulcan Activity Tracker will perform several actions both on the user interface side as well as the server side to perform application features. These actions will be designed for both processing efficiency and overall design cohesion. This outline serves as skeletal feature for brief guideline

of main components; however, developers must note that innerworkings will each be allotted with complex development effort.

PDL Outline

1. *Login* – student logs in with @pennwest.edu and application custom password
2. *Verification* – application entry upon verification
 - *Main Menu:*
 - student now has access to fitness tracker amenities and can navigate pages of the application through tabs and buttons
 - *Profile:*
 - student can make changes to profile or view current information
 - *Activities:*
 - student can log a fitness activity or view past activity
 - *Fields:* Activity Time, Calories Burnt, Location, Group Members, ect.
 - *Leaderboard:*
 - student can view leaderboard status for overall user participation
 - *Group Page:*
 - student can participate in group fitness activities by logging activities performed in group setting
3. *Database Entry:* Data sent from frontend, cleaned, then inserted into database
 - *Success: Code 200*
 - *Failure: Code 500*
4. *Logout-* student can close session cleanly with graphical logout button

Decision: Programming language / Reuse / Portability

Our backend code will be written in Python. We chose Python due to its ability to scale and security. Python's scalability is enhanced by asynchronous runtimes, Just-In-Time compilers, and Container orchestration platforms. While Python cannot match the speed of a raw language like C++, real-world systems care more about throughput and the ability to scale horizontally (Lamanna, 2025). Python's security features include static analyzers, frameworks like Django with built-in defenses against SQL injections, and a provided Open Web Application Security Project (OWASP), which maintains robust Python-specific guidelines and cheat sheets, essentially offering a blueprint for secure coding practices (Lamanna, 2025).

The backend code will also consist of a Microsoft SQL Server. This will be used for our database. A Microsoft SQL Server, or MSSQL, is a relational database management system (Microsoft 2025a). We chose MSSQL due to its compatibility with Python. Python 3 contains a package, mssql-python and dotenv, that can be installed easily in a command prompt. Then libraries from those packages, load_dotenv and connect, can be imported into the .py file. Create a SQL database, then create a .env file and add an entry for the connection string and begin creating the database (Microsoft, 2025b).

Implementation Timeline

The following table represents the timeline of implementation for the Vulcan Activity Tracker. This timeline takes into account initial software design through completion of manual, documentation, and final product presentation. As seen below, January through April is reserved for primary development time. April through May will be used for project validation, revisions,

testing, and final product presentation. This document serves as a contractual agreement for a defined timeline proposal.

<i>Activity</i>	<i>January</i>	<i>February</i>	<i>March</i>	<i>April</i>	<i>May</i>
<i>Component Design</i>					
<i>Software Design</i>					
<i>Unit Testing</i>					
<i>System Integration</i>					
<i>System Validation</i>					
<i>Operation Manuals</i>					
<i>Revisions/ Upgrades</i>					
<i>User Demonstration/ Presentation</i>					

Design Testing

To ensure the highest quality deliverable, this document was carefully revised. Team discussions were held to unite project scope, actions, and development path. Each team member added both valuable insight and documentation to this Design Document. Multiple drafts were formed, discussed, and revised to produce this final document. Digital communication was key in this process since it was developed primarily remote, however, in-person meetings were held to discuss specifics and hold progress updates.

The same procedure will be implemented during the software development phase of the Vulcan Activity Tracker. During the development process, each component and feature will be tested for quality of use, error checking, data validation, and usability. After timeline stages, the

project will also be tested for overall design cohesion. The development team will serve as primary testers; however, external testers will be implemented for egoless design opinions. Late-stage revisions and changes will be discussed as group and voted upon. Any design or development disagreement will be further held to an overall vote and gaining advice from professor. Overall, the development team and selected testers will brute force test the Vulcan Activity Tracker to the best of their ability to achieve high quality final product.

List of Resources

IBM. (2025, Feb 14th). *SQL error codes (-nnnn)*.

<https://www.ibm.com/docs/en/idr/11.4.0?topic=sm-sql-error-codes-nnnn>

Lamanna, E. (2025, June 6th). *Why Enterprises Are Still Choosing Python For Backend Development*. <https://dev.co/python/backend-development>

Microsoft. (2025a, May 19th). “*What Is SQL Server? - SQL Server.*” *Microsoft.com*

<https://learn.microsoft.com/en-us/sql/sql-server/what-is-sql-server?view=sql-server-ver17>

Microsoft. (2025b, November 18). *Quickstart: Python SQL Driver - mssql-python - Python driver for SQL Server. Microsoft.com*. [https://learn.microsoft.com/en-](https://learn.microsoft.com/en-us/sql/connect/python/mssql-python/python-sql-driver-mssql-python-quickstart?view=sql-server-ver17&tabs=windows%2Cazure-sql)

[us/sql/connect/python/mssql-python/python-sql-driver-mssql-python-quickstart?view=sql-server-ver17&tabs=windows%2Cazure-sql](https://learn.microsoft.com/en-us/sql/connect/python/mssql-python/python-sql-driver-mssql-python-quickstart?view=sql-server-ver17&tabs=windows%2Cazure-sql)

Mozilla (2025). *HTTP response status codes*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

Render. (2025). *Pricing*. <https://render.com/pricing>

Appendix I: Schematic and Bill of Materials

Cost constraints will be related to software toolsets and subscriptions. Development tools will need to be evaluated and considered with cost analysis. Since most toolsets and libraries will be open-sourced, the primary cost constraint for the Vulcan Activity Tracker will be web application hosting service. Most services only allot certain time periods for free hosting, or certain qualities of service. The best hosting application will be decided through team decisions.

Possible Hosting Service Options:

Render Pricing Sheet:

Hobby	Professional	Organization	Enterprise
For personal projects and small-scale applications.	For teams building production applications.	For teams with higher traffic demands and compliance needs.	For enterprises with critical security, performance and support needs.
\$0 USD (see usage/month plus compute costs)	\$19 USD (see usage/month plus compute costs)	\$29 USD (see usage/month plus compute costs)	Custom pricing
Start deploying	Select plan	Select plan	Get in touch
Deploy full-stack apps in minutes	Everything in Hobby, plus:	Everything in Professional, plus:	Everything in Organization, plus:
Fully managed databases	500 GB of bandwidth included	1 TB of bandwidth included	Centralized team management
Custom domains	Collaborate with 10 team members	Unlimited team members	Guest users
Global CDN & regional hosting	Unlimited projects & environments	Audit logs	SAML SSO & SCIM
Get security out of the box	Horizontal autoscaling	SOC 2 Type II certificate	Guaranteed uptime
Email support	Test with preview environments	ISO 27001 certificate	Premium support
	Private link connections		Customer success
	Isolated environments		
	Chat support		

Source: Render (2025)

Schematic of Toolset:

<i>Tool:</i>	<i>Price:</i>
MMSQL Management Studio	Open Source
VS Code	Open Source
Flask Framework	Open Source
HTML/CSS	Open Source
Python	Open Source

Appendix II: Team Details

This design document was developed through the collaboration of all team members. The document was made with the combined efforts of the entire team over the course of the project timeline. While each member contributed across various parts of the document, the primary contributors for each section are listed below:

- Margo Bonal:
 - Title Page, table of contents, Block System Description/Diagrams, Design details, architecture diagram, cohesion, coupling, design analysis, files accessed, Realtime requirements, messages/tables, implementation timeline/diagram, design testing, Appendix I-IV, List of resources, formatting, citations
- John Gerega:
 - Function Descriptions, Abstract, Decision: Programming Languages/ Reuse / Portability, Description of the Document
- Luke Ruffing:
 - Data Flow, Module Cohesion, Module Coupling, Detailed Tabular Description of Classes/Objects

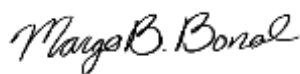
The rest of the major sections of the document were completed collaboratively with all members participating equally in discussions, decision making, and writing. This collaborative approach taken to completing the document allowed the group to refine ideas and reach a consensus before making any big decisions.

In addition to the writing of each section, all team members took part in proof reading and formatting ensuring the creation of a clear, concise and well-organized document. Together the team worked to check for proper grammar, proper citation, and adherence to all formatting guidelines.

Appendix III: Workflow Authentication

I, Margo Bonal, confirm that all work I have contributed to this document has been to the best of my knowledge, completed with integrity and in accordance with the objectives and collaborative standards established by the team. I attest that I will maintain these standards in all future contributions to this project.

Signature:



Date: 12/9/25

I, John Gerega confirm that all work I have contributed to this document has been to the best of my knowledge completed with integrity and in accordance with the objectives and collaborative standards established by the team. I attest that I will maintain these standards in all future contributions to this project.

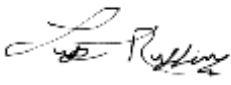
Signature:



Date: 12/9/25

I, Luke Ruffing, confirm that all work I have contributed to this document has been to the best of my knowledge completed with integrity and in accordance with the objectives and

collaborative standards established by the team. I attest that I will maintain these standards in all future contributions to this project.

Signature: 

Date: 12/9/25

Appendix IV: Report from Writing Center

We reviewed your Computer Science Senior Project Design document. You guys seemed to have everything that the rubric was requiring, double check that all the font and spacing is consistent and that you have 30 pages minimum (if you include the two evaluation pages and the title page then it should result in 33 pages). Your resources page should be alphabetical and the citation entry should be consistent, refer to your other documents for that. When you do in-text citations, for example Lamanna, don't italicize the text. We discussed semantics like frontend, front-end, front end; and smaller things like double spacing between words but that wasn't a continuous issue. When we get to the bullet points there were a few things that stood out: some titles had semicolons while others did not, a bullet point (now fixed) was too far over, titles were farther over than they began with, and some ended in periods while others did not. Remove the slashes near "Activity tracker" and "Backend/Server." Finally, as you pointed out you are trying to keep a consistent format with the spacing between titles and paragraphs, so later on in the paper there is the "Messages" section that should have a space included. Other than that the paper was very descriptive and seemed to have everything that the rubric was asking for and was well explained. Let me know if you have any further questions about the paper or need clarification on any part we discussed.